Family Tree Project Report

1. Project Overview

The **Family Tree** project is a C++ console application designed to manage and display relationships among family members. Using data structures such as **unordered_map** and **vectors**, the program stores information about each person, their spouse, and children. It supports queries to retrieve specific relationships, determine descendants, and analyze family connections.

---

2. Objectives

* Store and manage detailed information for family members.
* Display couples who have no blood relation.
* Retrieve relatives of a specified type (children, parents, siblings).
* Identify the person with the most descendants.
* Estimate the closest relation between two family members.

---

3. Data Structure Design

Person Structure

Each person is represented using a `struct` with the following fields:

* `id`: Unique identifier
* `firstName`, `lastName`

* `gender`

* `birthDate`, `deathDate`

* `spouseId`: ID of the spouse (-1 if no spouse)

* `childrenIds`: List of IDs of children

FamilyTree Class

The `FamilyTree` class manages all operations:

* **Storage:** `unordered_map<int, Person>` for O(1) access by ID.

* **Operations:** Adding people, linking children, displaying information, computing descendants, and analyzing relationships.

---

4. Functionalities

4.1 Display Couples Without Blood Relation

This function lists all couples. In this simplified version, all couples are assumed unrelated:

```cpp
void displayCouplesWithoutRelation();
```

4.2 Display Relatives of a Specific Type

Allows querying for `child`, `parent`, or `sibling` of a person:

```cpp
void displayRelatives(int personId, const string& relation);
```

4.3 Find Person with Most Descendants

Recursively counts all descendants of each person and identifies the one with the highest count:

```cpp
int countDescendants(int personId);
void personWithMostDescendants();
```

4.4 Closest Relation Between Two Persons

A simplified placeholder function that could be extended with advanced graph algorithms to find the precise familial relationship:

```cpp
void closestRelation(int id1, int id2);
```

---

5. Example Usage

Sample data used in `main()`:

```cpp
```

```
Person p1 = {1, "Ali", "Ahmadi", "Male", "1970-01-01", "", 2};
Person p2 = {2, "Sara", "Ahmadi", "Female", "1972-05-10", "", 1};
Person p3 = {3, "Reza", "Ahmadi", "Male", "1995-03-03", "", -1};

tree.addPerson(p1);
tree.addPerson(p2);
tree.addPerson(p3);

tree.addChild(1,3);
tree.addChild(2,3);
```

Sample Output:

```
Couples without blood relation:
ID: 1 Name: Ali Ahmadi Gender: Male
ID: 2 Name: Sara Ahmadi Gender: Female
------
Relatives of type child for Ali:
ID: 3 Name: Reza Ahmadi Gender: Male
Person with most descendants:
ID: 1 Name: Ali Ahmadi Gender: Male
Total descendants: 1
Closest relation (simplified output):
Ali and Reza might be distant relatives (advanced computation needed)
```

---

6. Project Limitations

* Blood relation verification is not implemented.
* Sibling detection is simplified and may not work for complex trees.
* Closest relation computation is a placeholder.
* No graphical visualization; all output is console-based.

---

7. Future Improvements

* Implement full blood-relation verification using graph traversal algorithms.
* Add GUI or web-based visualization.
* Support importing/exporting family data from files (JSON, CSV).
* Enhance relationship detection with exact familial distance calculation.

---

8. Conclusion

This project demonstrates the use of **C++ data structures** and **object-oriented programming** to model family trees and perform queries on relationships. While simplified, it lays a strong foundation for more complex genealogical systems.